



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 7

1. ENUNCIADO

Implemente um programa em C que crie uma lista encadeada de nós, cada um contendo o nome de um arquivo fictício. Dentro de uma região paralela, percorra a lista e crie uma tarefa com `#pragma omp task` para processar cada nó. Cada tarefa deve imprimir o nome do arquivo e o identificador da thread que a executou. Após executar o programa, reflita: todos os nós foram processados? Algum foi processado mais de uma vez ou ignorado? O comportamento muda entre execuções? Como garantir que cada nó seja processado uma única vez e por apenas uma tarefa?

2. DISCUSSÃO

Na biblioteca OpenMP, além das diretrizes criadas para evitar o "race condition" (condição de corrida), como `critical`, `atomic` e `reduction`, existem diretrizes ligadas a coordenação de fluxo de execução e distribuição de tarefas entre as threads, são estas: `task`, `single`, `master`, `barrier`, `nowait` e `taskwait`.

2.1. `#pragma omp task`

Uma unidade de trabalho é criada por uma thread, que será executada mais tarde por outra thread.

2.1.1. Diferença entre `omp for` e `omp task`

No `omp for`, a iteração é dividido antecipadamente dependendo da cláusula (`static`, `dynamic` e `guided`) entre as thread antes de iniciar a iteração, sendo ideal para processos envolvendo arrays, vetores e matrizes, já que são estruturas regulares; já no `omp task`, a tarefa é criada por uma thread e

depois executada por outra ou a mesma, o que é ideal para estruturas irregulares como processos envolvendo recursão, listas, árvores e grafos.

2.1.2. Shared vs. Private

Para evitar o race condition e ordem de execução imprevisível, pode ser usado referências específicas como o private ao se criar a task para garantir que cada tarefa processe o nó certo.

- shared: compartilhada por todas as threads; pode causar race condition se houver escrita simultânea.
- private: cada thread tem sua própria cópia, evitando conflitos de acesso.

2.1.3. Execução não determinística

As tasks e threads têm ordem de execução não determinístico, isto é, mesmo com a mesma entrada e código, o sistema operacional pode escalar as threads de modo diferente, ou mesmo a temporização mudar.

Para garantir que cada nó seja processado uma única vez, deve ser criada uma task com uma referência estável, ou seja, não depender de uma variável de iteração que vai continuar mudando e usar uma cópia apropriada do ponteiro ao criar a task.

2.2. Single e master

Um ponto a ser discutido é qual thread vai criar a task, já que nessa situação paralela, podem ser criadas mais de duas tasks para o mesmo nó. Dessa maneira, podem ser usadas as diretrizes single e master para solucionar essa problemática.

2.2.1. #pragma omp single

Uma única thread é escolhida para percorrer toda a lista e criar as tasks, enquanto as outras ajudam a executar as tasks.

Por padrão, existe uma barreira implícita no final do bloco single, fazendo com que as outras thread esperem, enquanto consomem e executam as tasks da fila, a thread percorrer toda a lista e criar as tasks.

2.2.2. #pragma omp master

A thread mestre (thread 0) é escolhida para percorrer toda a lista e criar as tasks, enquanto as outras executarão as tasks.

Neste caso, não existe uma barreira implícita no final do bloco master.

2.3. Flag

```
gcc -fopenmp tarefa7.c -o tarefa7
```

Pra variar o número de threads:

```
OMP_NUM_THREADS=4 ./tarefa7
```

```
OMP_NUM_THREADS=8 ./tarefa7
```

2.4. Etapas Para Resolução Da Tarefa

1º) Definir a struct No e ponteiro proximo;

2º) Criar função para inserir nós na lista;

3º) colocar a lista com nomes fictícios (ex: 10 arquivos);

4º) Abrir região paralela com `#pragma omp parallel`;

5º) Dentro dela, usar `#pragma omp single` para uma thread só percorrer a lista;

6º) Dentro do while, criar `#pragma omp task` com `firstprivate` no ponteiro do nó;

7º) Na task, imprimir nome do arquivo + `omp_get_thread_num()`;

8º) Liberar a memória da lista no final.

2.5. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.6. OUTPUT

Segue no apêndice.

3. CONCLUSÃO

Na implementação final, todos os dez nós da lista foram processados, cada um exatamente uma única vez, sem duplicações ou omissões, e distribuídos entre diferentes threads do sistema. No entanto, observa-se que a ordem de execução das tasks varia a cada execução do programa, refletindo a natureza não-determinística do escalonamento do OpenMP, comportamento esperado e que não compromete a corretude do resultado, já que o conjunto de nós processados permanece consistente. Para garantir que cada nó seja processado uma única vez e por apenas uma tarefa, foi necessário combinar três elementos: a criação da região paralela com ``#pragma omp parallel``, a restrição do percorrimento da lista a uma única thread por meio de ``#pragma omp single`` (evitando que todas as threads gerem tasks duplicadas para os mesmos nós) e a captura segura do ponteiro de cada nó com a cláusula

`firstprivate(atual)` (assegurando que cada task receba uma cópia própria do ponteiro no momento de sua criação, e não no momento de sua execução). Essa combinação foi suficiente para eliminar as condições de corrida observadas nas versões iniciais do programa.

4. APÊNDICE

4.1. RESOLUÇÃO

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

// Cada nó da lista encadeada guarda o nome de um arquivo e um ponteiro
// para o próximo nó
typedef struct No {
    char nome_arquivo[100]; // string com o nome do arquivo
    struct No* proximo; // ponteiro para o próximo nó da lista
} No;

// Recebe o ponteiro para o ponteiro da cabeça da lista (por isso No**),
// pois precisa modificar a cabeça da lista dentro da função
void inserir(No** cabeca, const char* nome) {
    // Alocar memória para um novo nó
    No* novo = (No*) malloc(sizeof(No));
    if (novo == NULL) {
        printf("Erro ao alocar memória\n");
        exit(1);
    }

    // Copia o nome do arquivo para o campo da struct
    strcpy(novo->nome_arquivo, nome);

    // O novo nó aponta para o que era a cabeça anterior
    novo->proximo = *cabeca;

    // A cabeça da lista agora é o novo nó
    *cabeca = novo;
}

// FUNÇÃO PARA LIBERAR A MEMÓRIA DA LISTA
void liberar(No* cabeca) {
    No* atual = cabeca;
    while (atual != NULL) {
```

```

        No* temp = atual; // guarda o nó atual
        atual = atual->proximo; // avança para o próximo
        free(temp); // libera o nó guardado
    }
}

// FUNÇÃO QUE SIMULA O PROCESSAMENTO DE UM ARQUIVO
// Apenas imprime o nome do arquivo e o ID da thread que executou
void processar(No* no) {
    printf("Arquivo: %s | Thread: %d\n",
        no->nome_arquivo,
        omp_get_thread_num());
}

int main() {
    No* lista = NULL;

    // Inserir 10 arquivos fictícios
    char nome[100]; // inseri na ordem inversa
    for (int i = 1; i <= 10; i++) {
        sprintf(nome, "arquivo%02d.txt", i);
        inserir(&lista, nome);
    }

    // #pragma omp parallel pra paralelizar a task
    #pragma omp parallel
    {
        // #pragma omp single para apenas uma thread execute o bloco abaixo
        #pragma omp single
        {
            No* atual = lista;

            // Percorre a lista sequencialmente, mas para cada nó é criada
uma task
            while (atual != NULL) {

                //firstprivate: ele faz uma cópia do valor de atual no
momento da criação da task. Sem isso, quando a task fosse executada depois,
atual já poderia estar apontando para outro nó (ou NULL)
                #pragma omp task firstprivate(atual)
                {
                    processar(atual);
                }

                // thread que está no single avança na lista e cria a
próxima task
            }
        }
    }
}

```

```

        atual = atual->proximo;
    }

    // Ao sair do single, há uma barreira implícita que espera
    // todas as tasks terminarem antes de a região paralela acabar
}

// free
liberar(lista);

return 0;
}

```

4.2. OUTPUT

```

PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa7> gcc -fopenmp tarefa7.c -o tarefa7
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa7> ./tarefa7
Arquivo: arquivo10.txt | Thread: 8
Arquivo: arquivo01.txt | Thread: 13
Arquivo: arquivo08.txt | Thread: 9
Arquivo: arquivo07.txt | Thread: 1
Arquivo: arquivo03.txt | Thread: 5
Arquivo: arquivo02.txt | Thread: 7
Arquivo: arquivo05.txt | Thread: 12
Arquivo: arquivo04.txt | Thread: 6
Arquivo: arquivo06.txt | Thread: 10
Arquivo: arquivo09.txt | Thread: 4

```